

LexRAG

Technical Architecture

A legal contract & case-law RAG platform — hybrid retrieval, cross-encoder reranking, citation-grounded refusal-aware generation, with quality enforced as a CI gate.

PROJECT TYPE	Self-directed portfolio project. Not a production deployment — the golden corpus is real, licensed contract text, not client data.
DOCUMENT	Technical Architecture Reference
PRIMARY SOURCE	<code>docs/architecture.md</code>
PREPARED	2026
STATUS	Sprint 5, Days 1–6 complete, plus Day 6 hardening

FastAPI

Qdrant

Elasticsearch

RAGAS

MongoDB

CI Quality Gate

How to read this document: every capability is marked either *Implemented*, *Partially met*, or *Not met* against the project's own documented acceptance thresholds (`docs/01-requirements.md` §7) — including the two criteria the system currently fails, stated plainly rather than rounded up to "done."

SECTION 1

System Architecture

LexRAG ingests PDF contracts and case-law documents, chunks and embeds them, and fans the write out in parallel to three stores: MongoDB (document metadata and provenance), Qdrant (dense vector search), and Elasticsearch (BM25 keyword search). A query runs dense and keyword retrieval in parallel, fuses the results with Reciprocal Rank Fusion, reranks the fused candidates with a cross-encoder, then generates a citation-grounded answer — or refuses, if the retrieved evidence doesn't support one.

Retrieval and generation quality are measured against a 30-case golden dataset and enforced as a CI quality gate (FR-12), not tuned by eyeballing outputs.

Pipeline Overview

Ingestion

ingestion/

- PDF loading, configurable chunking, full provenance metadata (document, section, page, chunk index)
- Fan-out write to MongoDB + Qdrant + Elasticsearch; a document isn't marked `status: ready` until every write is validated
- Duplicate detection via SHA-256 content hash — a byte-identical re-upload is skipped, not re-ingested



Hybrid Retrieval

retrieval/

- Parallel dense vector (Qdrant) + BM25 (Elasticsearch) search
- Merged via Reciprocal Rank Fusion ([retrieval/fusion/](#))
- Optional `document_ids` filter scopes retrieval to specific documents via native store filters



Reranking

retrieval/reranker/

- Cross-encoder ([bge-reranker-v2-m3](#)) reranks fused candidates
- `RERANK_INPUT_TOP_K` caps candidates entering the reranker — the dominant latency cost



Generation

generation/

- Citation-grounded prompting — cites only retrieved passages
- Explicit refusal when evidence is insufficient or the question needs unsupported synthesis



Evaluation

evaluation/

- Golden-dataset harness reports recall@K, precision@K, faithfulness, refusal accuracy, per-stage latency
- CI gate (FR-12) fails the build if thresholds regress

Architecture Diagram

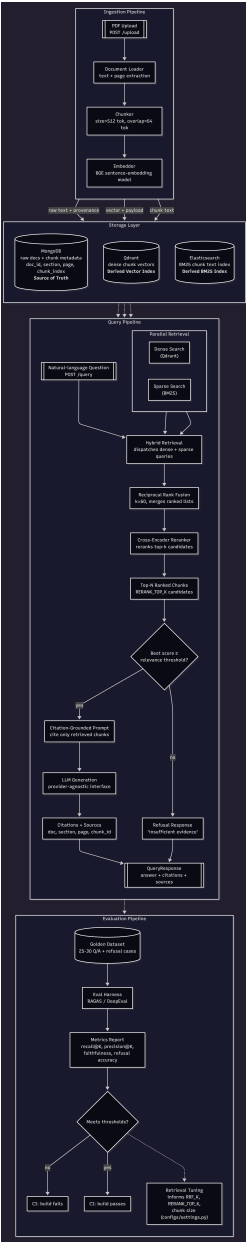


Fig. 1 — Ingestion pipeline, hybrid retrieval query pipeline, and evaluation pipeline gating CI. Editable source: docs/architecture.mmd .

SECTION 2

Component Responsibilities & Acceptance Status

Status is measured against the project's own documented thresholds (docs/01-requirements.md §7), not a general impression of completeness.

MET clears the documented threshold **PARTIAL** improved but not yet met **NOT MET** below threshold

Capability	Owns	Status
PDF ingestion + chunking + provenance	ingestion/loaders/ , ingestion/chunking/	MET
Dual-write consistency (Mongo/Qdrant/ES)	ingestion/pipeline.py — not ready until every write is validated	MET

Capability	Owns	Status
Duplicate detection (SHA-256)	<code>ingestion/repository.py</code>	MET
Hybrid retrieval (dense + BM25 + RRF)	<code>retrieval/hybrid.py</code> , <code>retrieval/fusion/</code>	MET — Recall@10 = 1.00 (≥ 0.85), Precision@5 = 0.88 (≥ 0.70)
Cross-encoder reranking	<code>retrieval/reranker/</code> — <code>bge-reranker-v2-m3</code>	MET
Citation-grounded generation	<code>generation/citations.py</code> , <code>generation/generator.py</code>	MET — Faithfulness = 0.98 (≥ 0.90)
Negative-case refusal (100% target)	<code>generation/prompts.py</code> — <code>LEGAL_RAG_V2</code>	NOT MET — 2 of 30 false acceptances remain
P95 query latency ≤ 6s (NFR-1)	Reranker is the dominant cost, CPU-bound	NOT MET — avg 26.3s, ~9x over budget
REST API (upload/query/documents)	<code>api/routes/</code>	MET
Golden-dataset evaluation harness	<code>evaluation/</code> — RAGAS + DeepEval	MET
CI evaluation quality gate (FR-12)	<code>.github/workflows/ci.yml</code> — <code>evaluation-gate</code> job	PARTIAL — real and wired, but manual-trigger + self-hosted only, not automatic on every push
Lint/type/test CI (<code>quality</code> job)	Ruff, mypy, pytest — every push/PR	MET

The `confidence` field is not an answer-correctness score.

`domain.generation.GenerationResult.confidence` is the cross-encoder's relevance score for the single highest-ranked retrieved chunk — the same score refusal (FR-10) gates on — not a calibrated measure of whether the generated answer is correct. A follow-up measurement (`scripts/confidence_correlation.py`) found Pearson $r \approx 0$ between `confidence` and answer correctness/faithfulness across the golden dataset, including one answer that scored 0.92 while being completely unfaithful. This is documented, current behavior — treat `confidence` as "how relevant was the best single piece of evidence," and rely on the `citations` list to judge whether an answer is trustworthy.

Module Ownership Boundaries

- `api/` — HTTP concerns only; parses requests, calls into ingestion/retrieval/generation, shapes responses
- `ingestion/` — PDF loading, chunking, the fan-out pipeline to all three stores
- `retrieval/` — vector store, keyword store, RRF fusion, reranker, each behind an interface
- `generation/` — citation-grounded prompting, provider-agnostic LLM interface, refusal logic
- `evaluation/` — golden-dataset runner + RAGAS/DeepEval metric wrappers
- `configs/` — all settings flow through here; nothing else reads `os.environ` directly

SECTION 3

Data Flow

Ingestion

- 1. `POST /upload` receives a PDF; its content hash is checked against existing documents — a byte-identical duplicate is skipped and the existing document returned.
- 2. The PDF is loaded, chunked with provenance metadata (document, section, page, chunk index), and embedded (`bge-m3`).
- 3. Chunks fan out to MongoDB (metadata), Qdrant (dense vectors), and Elasticsearch (BM25 index) in parallel.
- 4. The document is marked `status: ready` only once every store write is independently validated — a partial failure leaves it incomplete rather than falsely queryable.

Query

- 1. `POST /query` accepts a question and an optional `document_ids` scope.
- 2. Dense (Qdrant) and BM25 (Elasticsearch) retrieval run in parallel against the (optionally scoped) corpus.
- 3. Results merge via Reciprocal Rank Fusion, then the fused candidates (capped at `RERANK_INPUT_TOP_K`) are reranked by the cross-encoder.
- 4. The generator produces a citation-grounded answer from the top reranked passages, or refuses if they don't support one.
- 5. The response returns answer, citations, sources, and the reranker-relevance `confidence` field (see Section 2's callout on what it does and doesn't mean).

Request/Response Shape

`POST /query` → Hybrid Retrieval (Qdrant + Elasticsearch, parallel) → RRF Fusion → Cross-Encoder Rerank → Generation (cite or refuse) → JSON Response

SECTION 4

Technology Choices

Layer	Choice	Why
API	FastAPI	Async-first; I/O-bound store queries and LLM calls use async clients directly
Metadata / document store	MongoDB	Flexible schema for document metadata and provenance, independent of vector/keyword indexes
Vector store	Qdrant	Dense semantic retrieval; swappable behind a DI interface (Pinecone is a documented stretch goal)
Keyword index	Elasticsearch	BM25 keyword search — catches exact-term matches hybrid dense search alone can miss
Fusion	Reciprocal Rank Fusion	Combines dense + keyword rankings without needing score normalization across incompatible scales
Reranker	bge-reranker-v2-m3 (cross-encoder)	Refines fused candidates before generation; the dominant latency cost, so capped via <code>RERANK_INPUT_TOP_K</code>
Generation	OpenAI-compatible, provider-agnostic interface	Citation-grounded prompting isn't tied to one vendor's API shape
Evaluation	RAGAS + DeepEval	RAGAS supplies LLM-as-judge metrics; DeepEval wraps them in threshold/pass-fail assertions for CI
Package/env	uv	Fast, lockfile-based dependency management; <code>uv.lock</code> mismatch fails CI before any other check runs

Why an ONNX reranker backend was rejected

An ONNX Runtime backend for the same cross-encoder model was benchmarked as an alternative to the default PyTorch backend, specifically to address reranker latency (the pipeline's dominant cost). Measured result: 1.02x speedup — not a real improvement — so the added dependency and maintenance surface weren't adopted. Full write-up and numbers: `docs/adr/001-reranker-onnx-backend.md`. The latency win that did land came from a different lever: capping `RERANK_INPUT_TOP_K` (20 → 12), validated against the golden set with zero measured recall cost.

Why refusal is a generation-stage concern, not just retrieval

The two remaining negative-case false acceptances are not retrieval failures — both are cases where retrieval correctly returns real, topically-relevant evidence that doesn't actually contain the specific fact being asked about, and generation still answers instead of refusing. This is why the Day 6 fix (`LEGAL_RAG_V2`) targeted the generation prompt, not the retrieval or reranking stages.

SECTION 5

Error Handling & Consistency

Dual-Write Consistency

Ingestion writes to three independent stores (MongoDB, Qdrant, Elasticsearch) with no distributed transaction across them. A document is not marked `status: ready` in MongoDB until every store write is independently validated — so a partial failure (e.g. the Elasticsearch write fails after Qdrant succeeds) leaves the document in a non-ready state instead of silently becoming queryable with incomplete indexing.

Refusal as an Explicit Error Path

Generation is instructed to answer only from retrieved passages and to refuse explicitly — not answer with lower confidence — when the evidence is insufficient or the question requires synthesis the retrieved evidence doesn't support. Refusal is a first-class response, not an exception; it's measured in the same evaluation harness as recall, precision, and faithfulness (see Section 7 of the Case Study).

Design principle: a document is never queryable in a partially-indexed state, and an answer is never presented without the evidence that supports it — both are enforced structurally, not by convention.

SECTION 6

Testing & CI

Unit tests never hit a real MongoDB/Qdrant/Elasticsearch/LLM — the client is mocked or faked at the interface boundary. Real-service tests live in `tests/integration/` under the `integration` pytest marker and skip themselves when the dependency is unreachable. `pytest` with no flags must pass with zero external services running and finish in under 60 seconds (NFR-5).

CI Job	Trigger	What it runs
<code>quality</code>	Every push / PR	<code>uv sync --locked</code> , <code>ruff check</code> , <code>ruff format --check</code> , <code>mypy</code> , <code>pytest</code>
<code>evaluation-gate</code> (FR-12)	Manual (<code>workflow_dispatch</code>), self-hosted runner	<code>make evaluate</code> + <code>make evaluate-gate</code> against the pre-seeded golden corpus

The evaluation gate isn't automatic on every push because the golden corpus (`data/raw/sample_contracts/`) is real, licensed contract text, deliberately gitignored — a GitHub-hosted runner has no way to reproduce it from a checkout alone. The job itself is real, tested YAML, demonstrated both failing (against the pre-fix baseline and against deliberately emptied vector/keyword stores) and passing (against the real, correctly-configured system) — not aspirational configuration.

Known Limitations

Honest, measured gaps against `docs/01-requirements.md` §7's acceptance bar — not resolved this sprint, tracked here rather than glossed over.

Limitation	Detail
100% negative-case refusal not met	2 of 30 golden cases are still answered when they should be refused — both generation-stage judgment failures on topically-real-but-non-answering evidence
P95 query latency not met	Cut 53% (56.4s → 26.3s avg), but the CPU-only cross-encoder reranker remains ~9x over the 6s budget; no GPU/DirectML path on the dev machine, and ONNX was measured and rejected
Evaluation gate not automated on GitHub-hosted runners	Real, correctly wired, but only runs on a self-hosted runner with the (deliberately gitignored, licensed) corpus pre-seeded, triggered manually
Retrieval robustness to config mistakes untested at scale	Only an artificially emptied vector/keyword store cleanly failed the CI gate's checks on this 8-document corpus; a larger, more heterogeneous corpus would be needed to demonstrate a naturally occurring regression
Single-run measurements throughout	RAGAS/DeepEval LLM-judge scores aren't perfectly deterministic run-to-run — a repeat run of an identical config previously shifted refusal accuracy from 93.33% to 100%

Resources

Want to verify or explore this further? Here you go.

GitHub repository	github.com/Terrytd0/LexRag	README	README.md
Full source, commit history, CI configuration		Setup, features, benchmark numbers, known limitations	
Evaluation notes	docs/experiments/evaluation_notes_day6.md	Requirements	docs/01-requirements.md
Full Day 6 methodology, every intermediate measurement		Full functional/non-functional requirements & §7 acceptance criteria	
Case Study	case-study.pdf	Full docs folder	docs/
The companion document to this architecture reference		Architecture ADR, requirements, experiment notes	